

Attention-based multi-layer convolutional GRU for on-site glucose prediction on smartphone

Ece Karaalp^a, test^b and test^{b,*}

^aDepartment of Software Engineering, Izmir Bakircay University, 35620 Izmir, Turkey.

^bDepartment of Electrical and Electronics Engineering, Izmir Katip Celebi University, 35620 Izmir, Turkey.

ARTICLE INFO

Keywords:
Glucose Prediction
Smartphone Application

ABSTRACT

1
2
3
4
5
6
7
8
9
10

1. Introduction

Diabetic retinopathy (DR) is a major microvascular complication of diabetes and a leading cause of preventable vision loss worldwide. The disease often progresses asymptotically until advanced stages, making early detection essential for preserving sight and reducing long-term disability. Five-stage severity grading is clinically meaningful because it aligns with referral urgency, treatment planning, and follow-up schedules, and it helps distinguish patients who require immediate specialist intervention from those who can be managed in routine monitoring pathways. Accurate grading is also important for longitudinal monitoring, where small changes in lesion burden can alter surveillance intervals. In practice, ocular conditions frequently co-occur; patients with diabetes may simultaneously present with other retinal or optic nerve diseases, and coexisting pathologies can influence risk stratification and clinical decision-making. These realities motivate automated, fundus image-based systems that aim to deliver consistent, scalable, and objective assessments in screening programs, especially in settings with limited specialist availability. The prevailing methodological landscape relies on convolutional neural networks (CNNs) that learn discriminative representations directly from fundus photographs. Transfer learning, commonly using backbones such as DenseNet, ResNet, and EfficientNet, addresses data scarcity by leveraging pretraining on large-scale image corpora. To stabilize training and standardize image quality, preprocessing steps such as contrast enhancement, normalization, and illumination correction (including CLAHE) are frequently applied. Data augmentation and imbalance handling strategies are used to improve generalization and mitigate skewed class distributions. Within this context, DR screening has evolved from binary

detection to multi-class grading, while multi-label classification has emerged as a parallel line of research for predicting coexisting ocular diseases from the same fundus image, reflecting a broader shift toward comprehensive ocular risk profiling.

Despite methodological maturation, several limitations persist. Early-stage DR detection remains unstable, especially at the boundary between no DR and mild DR, where lesions are subtle and image quality is variable. Severe and proliferative classes are typically underrepresented, leading to pronounced class imbalance in datasets such as APTOS 2019 and consequently inconsistent minority-class recall. These issues are compounded by inter-grader variability and the ordinal nature of DR severity, which can cause small prediction errors to have significant clinical implications. Many pipelines depend heavily on aggressive enhancement or handcrafted preprocessing, which can introduce artifacts, amplify noise, or reduce robustness across acquisition conditions. Furthermore, models trained and tuned on a single dataset often generalize poorly to different populations or imaging devices, limiting real-world transferability. From a clinical perspective, DR grading is commonly treated as an isolated task, while multi-label ocular disease prediction is handled in separate studies with distinct objectives and evaluation protocols. This separation limits the ability to build integrated decision-support systems that concurrently grade DR severity and estimate the risk of other ocular comorbidities in a unified, clinically coherent manner.

Earlier studies on APTOS 2019 have addressed these challenges through complementary strategies. Transfer learning with DenseNet121 has shown strong overall accuracy and high quadratic weighted kappa (QWK), indicating that densely connected feature reuse can capture ordinal severity transitions, although class-wise stability varies. Hybrid feature fusion approaches that combine handcrafted descriptors such as Histogram of Oriented Gradi-

 volkan.kilic@ikcu.edu.tr (test)

ents with CNN features have attempted to improve lesion sensitivity, offering incremental benefits in certain metrics without fully resolving fine-grained class separation. Enhancement-driven pipelines employing CLAHE, sometimes coupled with super-resolution methods such as ESR-GAN, have aimed to amplify microaneurysms and hemorrhages, reporting improved headline accuracy but with sensitivity to preprocessing choices. Imbalance-aware training strategies, including synthetic oversampling (e.g., SMOTE) combined with CLAHE and CNN backbones, have achieved high average accuracy, yet the risk of distribution shift and inconsistent minority performance remains. Dual-branch global–local architectures have introduced complementary representations of retinal context and lesion-focused detail, improving ordinal consistency at the cost of added complexity. EfficientNet-based models with class-balanced augmentation have demonstrated competitive performance while emphasizing parameter efficiency. Some studies also report interpretability analyses using saliency or attention visualizations, which can help validate lesion localization but are often evaluated qualitatively rather than embedded into the learning objective. Collectively, these studies demonstrate that enhancement, augmentation, and architectural innovations can improve aggregate metrics, but they also reveal trade-offs between overall accuracy and class-wise robustness.

However, important gaps remain. Many studies optimize a single axis—such as preprocessing, imbalance handling, or backbone selection—without integrating these dimensions into a cohesive, clinically oriented framework. DR grading is most often examined as a standalone classification task, while multi-label ocular disease prediction is typically evaluated independently. As a result, the field lacks a unified modeling perspective that can simultaneously address five-class DR severity grading and the broader spectrum of ocular comorbidities. Balanced class-wise performance, especially for minority classes, remains unstable across settings, and integrated decision-support considerations are seldom addressed, including how outputs could be combined to support triage decisions and downstream referral pathways. Consequently, there is no widely adopted framework that jointly targets DR severity grading and multi-label ocular risk estimation within a single conceptual structure suitable for clinical decision support.

In this study, we address these limitations by developing a unified deep learning framework that spans two complementary tasks. First, we perform five-class DR grading using the APTOS 2019 dataset, currently employing a DenseNet121-based backbone as a configurable component that can be updated in future work. Second, we extend the framework to multi-label ocular disease prediction using the ODIR-5K dataset, treating it as an extensible module that will be systematically compared with existing multi-label ocular disease studies in future extended work. The framework is evaluated with a multi-metric strategy, including Accuracy, Precision, Recall, F1-score, and ROC-AUC, to capture both overall effectiveness and class-wise behavior and

to discourage reliance on a single aggregate metric. Rather than claiming state-of-the-art performance, the contribution is positioned as a methodological consolidation that integrates stage grading and multi-disease risk modeling under a clinically meaningful decision-support perspective, with an extensible design that supports future backbone updates and broader comparative analyses. The unified framing is intended to clarify how DR severity and comorbidity signals can be interpreted together in clinical screening workflows, while leaving room for future empirical optimization.

This paper is organized as follows: Section 2 presents the materials and methods, Section 3 describes the experimental setup and evaluation strategy, Section 4 reports the results and discussion, and Section 5 concludes the study.

2. Related Works

Research on DR severity grading with APTOS 2019 has primarily advanced along several methodological tracks. Transfer learning dominates the literature, with DenseNet-, ResNet-, and EfficientNet-based backbones widely adopted to leverage pretraining for robust feature extraction on limited medical datasets. These approaches often report strong overall accuracy and competitive QWK values, underscoring the value of pretrained representations for ordinal grading. Enhancement-based pipelines represent a second major direction; techniques such as CLAHE and normalization are used to standardize image appearance and highlight subtle lesions, and more aggressive enhancement strategies incorporate super-resolution to emphasize fine-grained pathology. While these methods can improve aggregate metrics, their performance is often sensitive to preprocessing parameters, and gains in headline accuracy do not always translate to stable class-wise sensitivity. A third direction focuses on imbalance-aware strategies, including reweighting, resampling, and synthetic oversampling. These techniques aim to mitigate skewed class distributions in APTOS 2019, with reported improvements in minority-class recall but persistent variability across severe classes. Finally, dual-branch and hybrid models attempt to integrate global retinal context with lesion-focused detail or to fuse handcrafted descriptors with CNN features, offering improved interpretability and ordinal consistency at the expense of increased architectural complexity.

Multi-label ocular disease prediction has developed as a related but distinct research area, emphasizing the need to detect coexisting conditions from a single fundus image. General deep learning approaches for multi-label classification typically rely on CNN backbones with sigmoid-based outputs and loss functions tailored to label imbalance, sometimes incorporating attention mechanisms or localization cues. However, detailed comparisons specific to ODIR-5K are beyond the scope of the present study because a systematic review of that dataset has not yet been completed. This paper therefore treats multi-label ocular disease prediction as an extensible component of the unified framework, and a comprehensive ODIR-5K comparative evaluation will

be integrated in extended future work.

3. Methods

This paper aims to predict blood glucose levels using the OhioT1DM dataset, which provides glucose measurements every 5 minutes. The paper focuses on predicting glucose levels 15, 30, 45, 60, 90, and 120 minutes ahead. To accomplish this, the sliding window approach is used to use past glucose measurements as input features for the prediction. The sliding window approach is a common technique used in time series analysis to generate input data for predictive models. In this approach, a window of fixed size is slid across the time series data, and the values within the window are used as input features for the model. Input data and output data are represented by $[BG_1, \dots, BG_t]$, $[BG_{t+1}, \dots, BG_{t+N}]$ respectively. Where BG is the blood glucose value, t is the window size and N is the prediction horizon. This approach effectively captures temporal dependencies in the blood glucose data set, resulting in accurate predictions for different prediction horizons.

This section provides detailed information about the neural networks, convolution and attention layers used in the proposed approach and the mobile application designed for the end user.

3.1. Time-series encoder

3.2. Time-series decoder

In this paper, three different deep neural networks, namely RNN, LSTM and GRU, were used for predicting blood glucose levels.

The RNN model is a deep neural network for processing sequential data. RNN structures can be categorized as one-to-one, one-to-many, many-to-one, and many-to-many. Thanks to persistent storage, the neural network has a memory that can improve itself using previous computations. The latent state vector is processed with the corresponding weight vector and bias value to generate the output. The gradient value of each output is based on the computations and processing in the previous step, which makes it difficult to train the RNN model. To overcome this problem, Backpropagation Through Time (BPTT) is proposed. This method limits the gradient and computational burden for each output in the previous time steps used in the time series. For long-term learning, exceeding the computational load limit with the BPTT method causes the problem of gradient explosion and disappearance. The gate mechanisms of LSTM and GRU, which contain memory and hidden state vectors similar to the RNN model, are proposed to solve these problems.

LSTM is a type of RNN architecture widely used in artificial intelligence for processing sequential data, such as video. Unlike traditional RNNs, LSTM addresses the vanishing gradient problem, which is a major challenge in training RNNs. LSTM introduces additional structures, including a cell and three gates, to allow the network to retain and use information from previous time steps. These structures

increase the complexity of the LSTM architecture but significantly improve its performance. The Forget Gate, one of the added components, filters out irrelevant information from the cell state. It combines the input from the previously hidden layer and the current input using the sigmoid function, which outputs values in the range $[0, 1]$. A value close to 0 indicates that the information should be forgotten, while a value close to 1 indicates that it should be retained. The Input Gate allows useful information to be added to the cell state. Similar to the Forget Gate, it uses the sigmoid function to determine what information should be retained. The resulting retention state is then combined with a vector of possible values, which is weighted and transformed using the tanh function. The tanh function outputs values between -1 and 1, making it suitable for binary classification tasks. The output gate determines the value to pass to the next layer. It involves passing the previous hidden state and the current input through the sigmoid function. The cell state is passed through the tanh function, and the two results are multiplied together to produce the "previous value", which is then sent to the next layer. Once the gate operations for the current time step are completed, the cell state for the next time step is determined. The cell state is responsible for propagating information throughout the network, both within the current cell and to other cells. The result of the Forget Gate is multiplied by the previous cell state and combined with the value from the Input Gate. After applying the sigmoid activation function, information with a value of 0 is discarded, while information with a value of 1 continues to flow through the cell state.

GRU is a different type of RNN architecture that provides a simpler alternative to LSTM while maintaining comparable performance in sequential data processing tasks. GRU achieves this simplification by combining the forgetting and input gates of LSTM into a single gate called the "update gate", and by combining the cell state and hidden state into a single "hidden state" representation. Compared to LSTM, GRU reduces the number of gating mechanisms and the complexity of computations, resulting in a more streamlined architecture that is easier to train and interpret. Despite these simplifications, GRU has been shown to be effective at capturing long-term dependencies in sequential data. In GRU, the update gate plays a critical role in controlling the flow of information. It determines how much of the previous hidden state should be mixed with the current input to produce the updated hidden state. The update gate takes as input the previous hidden state and the current input, and passes them through a sigmoid function. The output of the sigmoid function determines how much the previous hidden state influences the updated hidden state. In addition, GRU introduces a "reset gate" that determines how much of the previous hidden state should be forgotten when computing the updated hidden state. The reset gate takes the previous hidden state and the current input as inputs and applies a sigmoid function. The resulting output is used to reset the hidden state by determining how much of the previous hidden state to ignore. The updated hidden state is then computed

by combining the reset gate with the current input. The reset gate is used to determine which parts of the previous hidden state are considered relevant for the current time step. The updated hidden state is obtained by applying the tanh function to a combination of the reset gate, the previous hidden state, and the current input. This produces a new representation of the hidden state that includes both the current input and relevant information from the previous hidden state.

A convolutional layer is a type of layer commonly used in deep neural networks, particularly for processing data with spatial or sequential structures, such as images or text. The layer consists of a set of learnable filters, also called kernels or weights, applied to the input data in a sliding window fashion. The input data is then processed by each filter, which computes a dot product between its weights and the values in the corresponding window of the input. The output of this dot product is a single value, which is then passed through a nonlinear activation function, such as a rectified linear unit (ReLU), to introduce nonlinearity into the output. The convolutional layer produces a set of feature maps that capture different characteristics of the input data obtained through the application of multiple filters. These feature maps represent trends or cycles in sequential time data and allow the network to learn temporal dependencies.

Another layer of the proposed method, the attention mechanism, plays a crucial role in capturing the relationships and dependencies between different parts of the input data. An attention layer computes the importance or relevance of each element in a sequence with respect to the others. It assigns weights to different elements based on their importance, allowing the model to focus more on the relevant parts during processing. The process of calculating attention weights typically involves a comparison between a query vector (representing the current state of the model) and a set of key vectors (representing the input sequence). The result of this comparison is a score for each key vector, which is converted to a probability distribution using a softmax function. The resulting weights are then used to calculate the weighted sum of the key vectors that make up the context vector. The resulting context vector, obtained by the weighted sum of the key vectors, provides a comprehensive representation that captures the salient information from the input sequence, allowing the model to focus on the most relevant elements and make informed decisions or predictions based on the context.

3.3. Data Preprocessing

Before performing the training, several preprocessing steps were performed on the OhioT1DM dataset to ensure that the data was consistent and ready for training.

Feature selection is selecting a subset of relevant features (predictors, input variables, or attributes) from a larger set of features for use in model construction. The goal of feature selection is to identify the essential features that contribute most to the prediction or classification of the target variable while minimizing the number of features used. In this paper, The effect of feature number on prediction accuracy was in-

vestigated first. A correlation heatmap was used for feature selection. Correlation heatmaps are a type of graph used to visualize relationships between numerical variables. A correlation chart typically contains a set of numerical variables, with each variable represented by a column and the rows representing the relationship between each pair of variables. Values in cells indicate the strength of the relationship; positive values show a positive relationship and negative values show a negative relationship. Color coding of cells makes it easy to understand the relationships between variables. A separate heatmap graph was drawn for each patient to see the relationships between the features. According to the results, it was seen that the glucose values of patient 540 had a stronger relationship with 2 other features (carbohydrate and basal insulin).

When the data set was examined, it was seen that there were missing values in some parameters. And these values can cause the model to learn incorrectly. In order to prevent this, linear interpolation was applied to the training set for missing values whose duration is less than 30 minutes, as shown in Figure 2 (a) and (b). Interpolation is a mathematical method of generating new data points within a discrete set of known data points. In other words, it is estimating values between two known values. This can be done using polynomial, spline, and linear interpolation techniques. Linear interpolation is a method for predicting the value of a continuous variable at a point between two known values. Linear interpolation is a method of predicting the value of a function at a point that lies between two known points on the function. Given two points (x_0, y_0) and (x_1, y_1) on a function, linear interpolation allows us to predict the value of the function at a point x , where $x_0 \leq x \leq x_1$. The predicted value, y , is calculated by taking a weighted average of the two known values, y_0 and y_1 , with the weighting determined by the distance of x from the two known points. The equation of linear interpolation is:

$$y = mx + b \quad (1)$$

where, y is the interpolated value, m represents the slope of the line, x is the independent variable and b is the y -intercept of the line. In addition, missing time steps in data taken at discrete times, such as bolus insulin and carbohydrate, were filled with zero

Data normalization was used to ensure that the parameters in different value ranges, such as glucose, carbohydrate, and insulin, which are the identified attributes, contribute equally to the learning process. Data normalization overcomes the different types and sizes of numerical values in the data set, thereby improving the learning performance of the model. Standard scaling was used in this paper, which is a common normalization method that involves transforming the data to make the mean 0 and the standard deviation 1, resulting in a distribution that approximates a normal distribution. The equation of the standard normalization is as

./Figures/modelArticle_v2.png

Figure 1: Model Architecture.

Figures/graph1.png

Figure 2: The graph of the 8 NaN values in the unprocessed raw data dated 1 June was filled with linear interpolation.

follows:

$$z = \frac{(X - \mu)}{\sigma} \quad (2)$$

where, X represents the original value, μ represents the variables mean, and σ represents the variables standard deviation. The resulting z-value is the standardized value.

3.4. Proposed attention-based multi-layer Glucose Prediction

The proposed model utilises neural networks to predict future blood glucose values. In this regard, structures such as recurrent and convolutional neural networks are used. The main reason for this is that recurrent neural networks are effective in processing sequential data such as time series and one-dimensional convolutional network is successful in feature extraction in time series data. In addition to these networks, the attention mechanism was added to improve the learning of the model. The goal of the attention mechanism is to reduce larger, more complex tasks to smaller, more manageable attention domains in order to understand and process them sequentially. Using the attention mechanism allows the network to focus on a few specific aspects at the same time and ignore the rest. As input to the model, glucose values up to n time steps found at 5-minute intervals in the past are given. These time steps vary according to the prediction horizon of the model, for example, for 30 PH there are 6-time steps with 5-minute intervals. These time steps range from the current time (t) to the time 25 minutes before ($t-5$). The training of the model was carried out with different time steps using 15, 30, 45, 60, 90, and 120 PH. The input is first given to the convolutional layer, and since the input to this layer is a sequential time series, a one-dimensional convolutional network is used. In the convolutional network, the kernel size is set to 1 to obtain f pieces of feature maps (filters). In this layer, each element in the input array is subjected to the kernel dot product process and then passed through the RELU activation function, as a result of this process, f pieces of $1 \times n$ sized arrays are obtained. With this process, the features in the sequential data were found, then these f pieces of arrays are given as input to the GRU layer consisting of

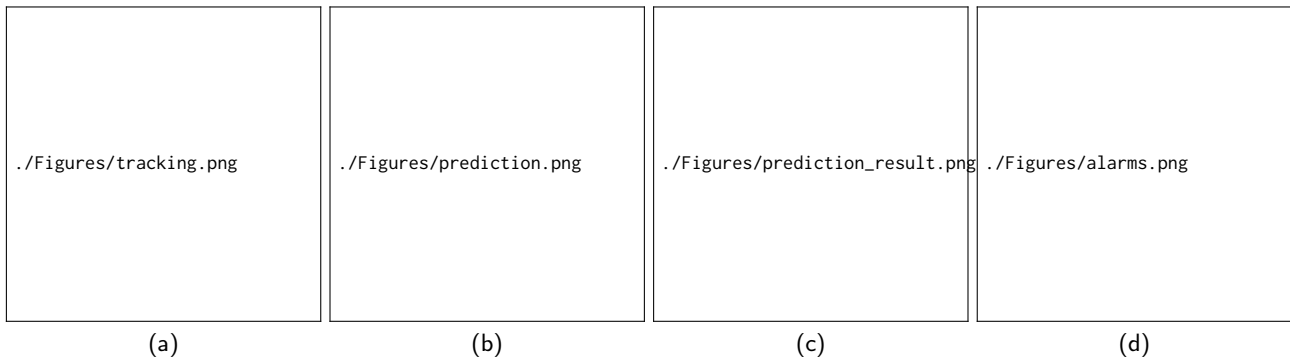


Figure 3: The screens of the GlucoWizard application can be listed as follows: (a) shows the tracking chart, (b) shows the prediction screen, (c) shows the post-prediction information screen, and (d) shows the alarm and reminder screen.

512 units. In order to improve learning in these layers, a multilayer structure is developed and 2 GRU layers with the same parameters are used, also the tanh activation function is used at the output of each layer. The arrays obtained from the output of the GRU layers are transferred to the attention layer, which has 500 units. Each output from the GRU layer enters the context vector as a global align weight and is processed here. The output obtained here, and the last output obtained from the GRU layer is given to the attention vector and processed here. The resulting output is connected to the dense layer, which contains n timesteps of neurons and uses a linear activation function. The output of the dense layer gives the estimated n glucose values. In other words, as a result of a model training with 30 PH, the output is glucose values consisting of 6-time steps ($t+1$, $t+2$, $t+3$, $t+4$, $t+5$, $t+6$).

3.5. Smartphone application: GlucoWizard

In this paper, an easy-to-use, large, and high-quality smartphone application was developed for people with diabetes to actively and easily access the developed prediction model. Thanks to the user-friendly interface of the application, individuals can predict future blood glucose values by entering past blood glucose values, perform retrospective checks, and receive reminder alarms for medication use and annual checkups.

The main features of the application consist of 3 main sections. The aim is to bring these sections together in a way that they complement and support each other. Type 1 Blood Glucose Prediction, Insulin/Blood Glucose Tracking Chart, and Alarms/Reminders are the three main titles.

The Type 1 Diabetes Blood Glucose Prediction screen, which is the first main screen of the application, allows users to make blood glucose predictions at 15, 30, 45, 60, 90, and 120 minutes. Figure 3 (a) visually shows the user-friendly interface where users can enter their past blood glucose values at 5-minute intervals. According to the predicted value, feedback is given as shown in Figure 3 (b) to take necessary precautions. The prediction models are converted into TFLite files and embedded into the mobile application. The embedded models allow users to predict blood glucose levels even when they are offline. The second main screen

of the application, the Blood Glucose Tracking Chart, was designed as the main page of the application so that users can enter their blood glucose readings regularly and conveniently throughout the day. The chart was designed to resemble the paper weekly tracking forms that diabetics are familiar with and use constantly, so that the patient can recognize and use the chart without feeling unfamiliar. The graphic design, familiar to the diabetic patient, presents the data of a 30-day measurement in a clear and understandable way. The user is presented with an interface where they can easily enter values and receive immediate feedback based on the values entered in the form of colored and text alerts, as shown in Figure 3 (c).

The alerts and reminders screen of the application is a critical tool for people with type 1 diabetes who rely on consistent medication adherence. The delicate balance of blood glucose control can be disrupted by missing doses of insulin or other medications. To address this challenge, the app provides customizable alarms, as shown in Figure 3 (d), allowing users to set reminders to ensure they never forget to take their medication.

In addition to medication adherence, regular medical appointments and check-ups play a critical role in the effective management of type 1 diabetes. But with the demands of a busy schedule, it's all too easy to overlook or forget these important appointments. The reminder feature of the application goes beyond alarms and allows users to create reminders for specific dates, ensuring they stay organized and never miss important medical appointments. By staying on top of their health routine and receiving timely guidance from healthcare professionals, individuals can prioritize their well-being and maintain optimal control over their condition.

4. Experimental Results

4.1. Dataset and performance metrics

The OhioT1DM dataset contains eight weeks of data for each of the 12 people with type 1 diabetes. These patients are referred to by randomly selected ID numbers 540, 544, 552, 559, 563, 567, 570, 575, 584, 588, 591, 596. All patients were on insulin pump therapy with CGM. They re-

ported their life-event data via a custom smartphone app and provided physiological data from a fitness band. The first cohort of six patients wore Basis Peak, and the second cohort wore the Empatica Embrace fitness bands. Data for the first cohort was released in 2018 for the first Blood Glucose Level Prediction (BGLP) Challenge, and data for the second cohort was released in 2020 for the second BGLP. The dataset includes a CGM blood glucose level every 5 minutes, finger sticks, bolus, and basal insulin doses, a self-reported meal with carbohydrate estimates, times of exercise, sleep, work, stress, illness, and different data from the fitness band. The data for individuals who wore the Basis Peak band includes 5-minute aggregations of heart rate, galvanic skin response (GSR), skin temperature, air temperature, and step count. The data for those who wore the Empatica Embrace band includes 1-minute aggregations of GSR, skin temperature, and magnitude of acceleration.

This paper used four statistical performance metrics and one clinical metric. Statistical metrics are RMSE, MAE, MAPE, R-squared, and the clinical metric is Clarke Error Grid. For testing the model, MAE and RMSE are commonly used in many systems to measure the difference between predicted and actual values of users. Both have been used for many years to evaluate model performance, but there is no objective judgment on the most appropriate metric for model errors.

$$RMSE = \sqrt{\frac{1}{m} \sum_{i=1}^m (X_i - Y_i)^2} \quad (3)$$

$$MAE = \frac{1}{m} \sum_{i=1}^m |X_i - Y_i| \quad (4)$$

MAPE is another performance metric for prediction problems but it is unsuitable for predicting problems where significant errors are expected. It is used in tasks where it is more important to be sensitive to relative than absolute variations.

$$MAPE = \frac{\%100}{m} \sum_{i=1}^m \left| \frac{X_i - Y_i}{Y_i} \right| \quad (5)$$

In the equations, X_i is represented as the predicted i th value, Y_i as the actual i th value, and m as the number of values.

The R-squared value, also known as the coefficient of determination, is a number between 0 and 1 which tells how much of the variance in the dependent variables can be explained by the independent variables in a regression model. It expresses the proportion of the total variance in the dependent variable that is predictable from the independent variables. A value of 1 means all the variance in the dependent variable can be explained, whereas a value of 0 means no variance in the dependent variable can be explained by the independent variables

$$R^2 = 1 - \frac{\sum_{i=1}^m (y_i - \hat{y}_i)^2}{\sum_{i=1}^m (y_i - \bar{y})^2} \quad (6)$$

Table 1

Performance comparison based on number of parameters and methods

Methods	Extensions	1-Parameter	2-Parameter	3-Parameter
RNN	Solo	13.42	13.45	13.45
	Bi	13.44	13.49	13.41
	Conv	13.35	14.11	13.96
GRU	Solo	13.44	13.54	13.51
	Bi	13.44	13.54	13.50
	Conv	13.20	14.54	14.38
LSTM	Solo	13.46	13.57	13.52
	Bi	13.45	13.51	13.49
	Conv	13.26	14.06	14.31

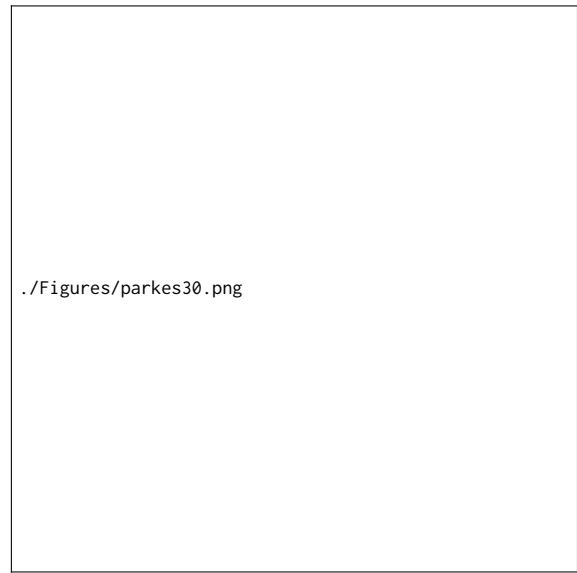


Figure 4: Parkes Error Grid of Patient 570 for 30 PH.

y_i is the observed value of the dependent variable $\hat{y}_i$ is the predicted value of the dependent variable $\bar{y}$ is the mean of the observed values of the dependent variable n is the number of observations.

In addition to statistical metrics, Clarke Error Grid metric, which is a clinical semi-quantitative tool, was used to evaluate the accuracy of glucose measurements in patients with diabetes. It plots the difference between a measured glucose level and the true glucose level on the x-axis, and the percent error on the y-axis. The grid is divided into five regions, each representing a different level of clinical significance. The "A" region represents measurements that are accurate, while the "B" region represents measurements that are clinically acceptable but may not be accurate. The "C", "D", and "E" regions represent measurements that are increasingly inaccurate and may lead to clinically significant errors in treatment decisions.

4.2. Results and discussion

1
2

Table 2

Performance comparison based on number of layers and attention

	Methods	Parameter	2-layer	4-layer	6-layer	8-layer	10-layer	12-layer
Without Attention	Conv + RNN	1	13.29	13.36	13.49	13.65	13.66	13.83
	RNN	2	13.49	13.47	13.65	13.80	13.96	14.04
	BiRNN	2	13.50	13.67	13.93	14.32	14.83	17.30
	RNN	3	13.56	13.75	13.94	13.99	13.97	14.12
	BiRNN	3	13.80	13.91	14.12	14.26	14.31	16.92
	Conv + GRU	1	13.18	13.16	13.23	13.31	13.53	13.70
	Conv + LSTM	1	13.27	13.24	13.47	13.71	13.77	24.51
	BiLSTM	2	13.52	13.46	13.64	13.80	14.05	14.23
	BiLSTM	3	13.53	13.66	13.87	14.00	14.29	14.42
With Attention	Conv + RNN	1	13.32	13.50	13.62	13.68	13.80	13.93
	RNN	2	13.37	13.51	13.68	13.82	13.93	14.06
	BiRNN	2	13.56	13.81	14.16	14.61	16.47	16.06
	RNN	3	13.64	13.78	13.93	14.04	14.07	14.20
	BiRNN	3	13.97	14.04	14.17	14.94	16.19	16.41
	Conv + GRU	1	13.13	13.20	13.29	13.29	13.73	13.76
	Conv + LSTM	1	13.22	13.25	13.42	13.74	13.84	28.86
	BiLSTM	2	13.43	13.46	13.64	13.98	14.10	14.28
	BiLSTM	3	13.49	13.87	14.10	14.21	14.54	14.72

Table 3

The errors according to PH and patients

	15 PH				30 PH				45 PH			
Patient ID	RMSE	MAE	MAPE	R2	RMSE	MAE	MAPE	R2	RMSE	MAE	MAPE	R2
540	8.04	5.40	3.72	0.99	13.91	9.22	6.58	0.96	20.02	13.47	9.73	0.91
544	6.94	4.75	3.36	0.98	11.67	7.76	5.24	0.95	16.50	10.80	7.16	0.91
552	6.25	4.16	3.22	0.99	10.75	7.16	5.63	0.96	15.11	9.91	7.60	0.92
559	8.02	5.05	3.31	0.99	12.96	8.01	5.28	0.96	18.27	12.01	8.46	0.93
563	9.62	5.58	3.73	0.96	13.27	8.33	5.36	0.92	17.13	11.11	7.15	0.86
567	7.56	4.89	3.51	0.98	13.39	8.33	5.94	0.94	19.09	12.39	9.37	0.87
570	6.70	4.24	2.18	0.99	10.85	6.88	3.54	0.97	14.71	9.50	4.99	0.95
575	10.09	5.39	3.98	0.97	15.94	8.91	6.32	0.93	20.31	12.05	8.61	0.88
584	9.51	6.15	4.12	0.98	15.11	9.85	6.62	0.94	20.55	13.60	8.94	0.89
588	7.88	5.38	3.29	0.97	12.83	8.69	5.42	0.93	17.43	11.71	7.20	0.87
591	9.53	5.96	4.59	0.96	14.88	9.65	7.39	0.91	19.90	13.20	10.02	0.84
596	7.59	4.60	3.36	0.98	11.73	7.41	5.42	0.95	16.34	10.80	8.02	0.90
Mean	8.14	5.13	3.53	0.98	13.11	8.35	5.73	0.94	17.95	11.71	8.10	0.89

Table 4

The errors according to PH and patients

	60 PH				90 PH				120 PH			
Patient ID	RMSE	MAE	MAPE	R2	RMSE	MAE	MAPE	R2	RMSE	MAE	MAPE	R2
540	25.75	17.15	12.44	0.86	34.69	24.10	17.31	0.74	42.66	30.65	22.55	0.60
544	20.16	13.19	8.76	0.86	26.91	18.76	12.86	0.75	33.06	24.21	16.73	0.61
552	19.47	12.91	10.08	0.87	26.20	17.73	13.97	0.77	32.03	23.11	18.32	0.65
559	23.65	16.47	12.10	0.88	30.29	20.33	13.37	0.81	37.33	26.52	18.15	0.71
563	20.95	13.97	8.81	0.79	28.06	18.93	11.98	0.63	31.58	22.29	14.33	0.54
567	23.93	15.64	11.54	0.80	32.64	21.72	15.39	0.64	38.33	27.18	20.52	0.51
570	18.70	12.54	6.62	0.92	24.21	16.59	9.43	0.86	30.72	22.10	12.74	0.78
575	26.52	16.31	12.14	0.80	34.23	23.32	18.22	0.67	38.55	27.44	21.38	0.57
584	24.86	16.57	11.01	0.84	32.73	22.82	14.98	0.72	38.39	27.33	17.72	0.61
588	21.67	14.71	9.25	0.80	27.66	19.21	12.06	0.67	32.85	23.08	14.41	0.54
591	23.01	15.65	12.23	0.79	29.16	20.93	17.01	0.67	34.19	24.91	20.18	0.55
596	20.03	13.31	10.00	0.85	27.10	18.42	13.68	0.72	31.03	21.51	15.94	0.64
Mean	22.39	14.87	10.42	0.84	29.49	20.24	14.19	0.72	35.06	25.03	17.75	0.61

Table 5

The performance comparison with further significant articles in the field

	15 Minutes		30 Minutes		45 Minutes		60 Minutes		90 Minutes		120 Minutes	
Method	RMSE	MAE	RMSE	MAE	RMSE	MAE	RMSE	MAE	RMSE	MAE	RMSE	MAE
LSTM + WaveNet + GRU	-	-	21.90	15.86	29.12	21.50	35.10	26.39	-	-	-	-
SVR	-	-	20.15	9.12	-	-	-	-	-	-	-	-
LSTM/ Linear Regression	-	-	19.83	14.09	-	-	33.63	25.34	-	-	-	-
Ensemble MMS and MMSZ	-	-	19.57	14.06	-	-	34.48	25.95	-	-	51.95	40.01
GluNet	-	-	19.28	-	-	-	31.83	-	-	-	-	-
Adversarial Transfer	-	-	19.27	8.68	-	-	-	-	-	-	-	-
LSTM	-	-	18.93	-	-	-	31.79	-	-	-	-	-
DRNN	-	-	18.90	-	-	-	-	-	-	-	-	-
XGBoost	-	-	18.90	-	-	-	-	-	-	-	-	-
CRNN	-	-	18.80	13.2	25.3	18.2	31.8	23.4	41.2	31.1	47.2	36.5
Fast-Adaptive Neural Network	-	-	18.64	13.25	-	-	31.07	22.86	-	-	-	-
Seq - Seq LSTM/ MLR	-	-	17.52	10.91	-	-	24.58	17.42	-	-	-	-
ARIMA / Holt-ANN	13.14	7.85	15.50	9.45	20.21	12.56	-	-	-	-	-	-
Proposed Method	8.14	5.13	13.11	8.35	17.95	11.71	22.39	14.87	29.49	20.24	35.06	25.03

3 10
4
5 1
6 2
7 3
8 4
9 5
10 6
1 7
2 8
3 9
4 10
5
6 1
7 2
8 3
9 4
10 5
1 6
2 7
3 8
4 9
5 10
6 1
7 2
8 3
9 4
10 5
1 6
2 7
3 8
4 9
5 10
6 1
7 2
8 3
9 4

5
6
7
8
9
10

1

2
3
4
5
6
7
8
9
10

5. Conclusion

1

2
3
4
5
6
7
8
9
10

Declaration of competing interest

Funding sources

Acknowledgements

References